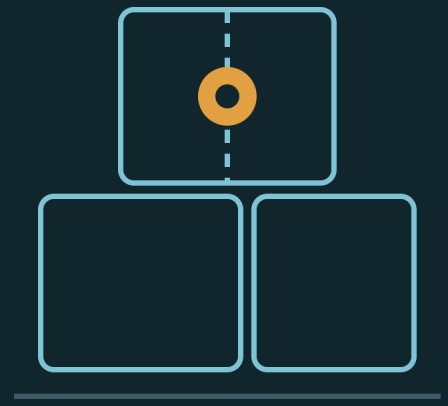
TP INTEGRADOR 2026 · TEMA 3 · GRUPO 95

Sistema de ventas e inventario



Del Centro de Distribución al mostrador: compras, logística, ventas y reportes

95294
Lenta, Martina

400501
Salomon Wendler, Thomas Ezequiel

400057
Maccio, Tomas Agustín

401939
Torrens Valsagna, Francisco

400254
Torres Baldo, Luis Agustin

Desarrollo de Aplicaciones con Objetos · UTN Facultad Regional Córdoba · Noviembre 2026

EL PROBLEMA

Un comercio con varios locales que se abastecen de un CD



> **«Nada debe ingresar a stock vendible sin verificación.»**
>
> La regla central del enunciado: la tomamos como invariante de todo el diseño.

ALCANCE

Siete procesos obligatorios

01

Locales y CD

Alta, edición y baja lógica; un único CD.

02

Catálogo

Productos, precios, mínimos y proveedores.

03

Reposición

Pedidos de locales en una orden CD → local.

04

Empaquetado

Bultos con remito y precinto de seguridad.

05

Despacho

Sale del CD y queda en tránsito.

06

Recepción

Control del precinto y doble verificación.

07

Ventas y devoluciones

Cobros, comprobante, stock y caja.

Extra

Compras

Ingresan stock al CD con su costo.

ARQUITECTURA

Las reglas viven en Python

`frontend/`

React + Vite

Captura datos y muestra respuestas. No decide reglas.

`api_*.py`

API Flask

HTTP/JSON: valida formato y responde 400, 404 o 409.

`dominio/`

Dominio POO

Clases, reglas, State y Strategy. No importa Flask ni SQLite.

`persistencia/`

SQLite

sqlite3, SQL parametrizado y una transacción por operación.

- Las validaciones definitivas se ejecutan en el backend, nunca solo en el navegador.
- El dominio se prueba sin servidor ni base de datos.
- Node solo compila el frontend; Flask sirve el build en la demo.

MODELO DE DOMINIO

Clases agrupadas por proceso

Catálogo

Catalogo
Categoria
Producto
Proveedor
Ubicacion

Compras

RegistroCompras
Compra
DetalleCompra

Logística

OrdenReposicion
Bulto
EstadoBulto (ABC)
EventoEstadoBulto
Remito

Ventas

RegistroVentas
Venta
DetalleVenta
Cobro
Comprobante

Devoluciones

RegistroDevoluciones
Devolucion
DetalleDevolucion
Reintegro

Stock y caja

Inventario
MovimientoStock
Caja
MovimientoCaja

Documentos confirmados

Compra, Venta y Devolucion se construyen completos, componen sus líneas y no se modifican.

Maestros con baja lógica

Nada se borra y un código dado de baja no se vuelve a usar.

Saldos con historia

Solo Inventario y Caja cambian saldos; cada cambio deja un movimiento inmutable.

PATRÓN STATE · CICLO DE VIDA DEL BULTO

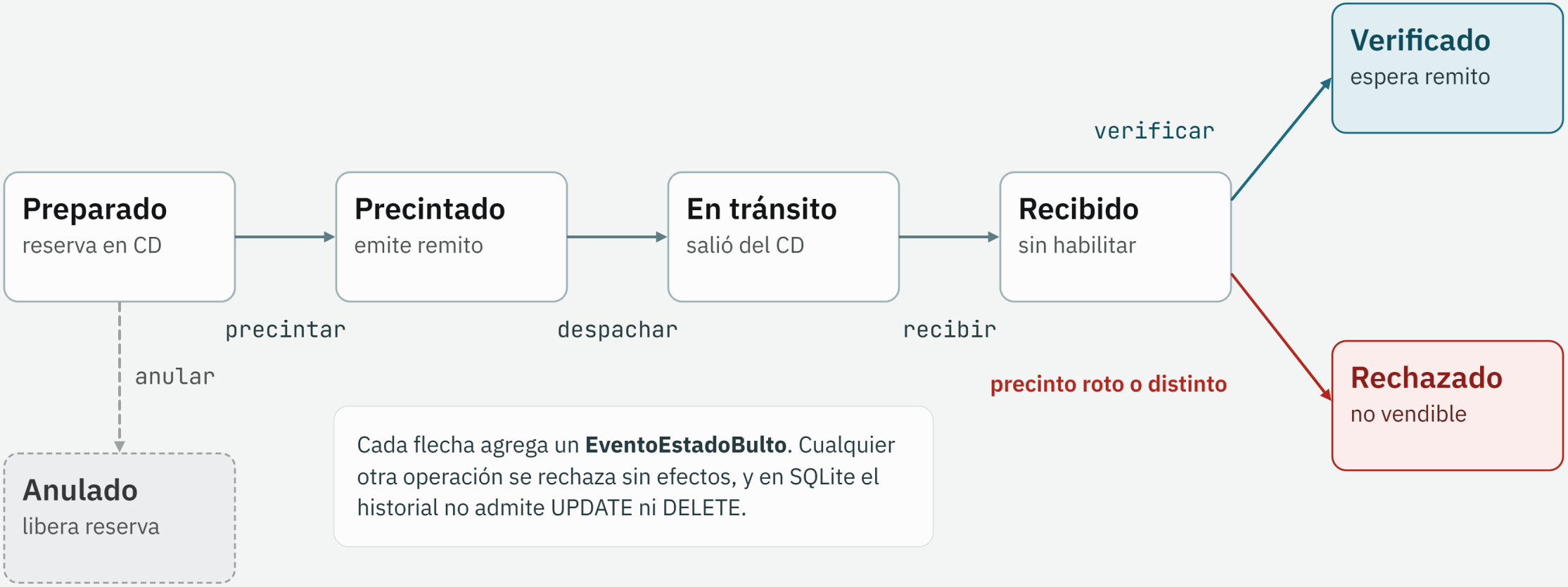
¿Qué puede hacer el bulto ahora?



El enunciado evalúa críticamente este ciclo: cada estado es una clase y cada cambio queda registrado para siempre.

PATRÓN STATE

Ciclo de vida del bulto



Cada flecha agrega un **EventoEstadoBulto**. Cualquier otra operación se rechaza sin efectos, y en SQLite el historial no admite UPDATE ni DELETE.

PATRÓN STATE · CÓDIGO

El bulto delega en su estado

```python
class EstadoBulto(ABC):
    def precintar(self, bulto, precinto, remito, instante):
        self._rechazar(bulto, "precintar")  # por defecto

class Preparado(EstadoBulto):
    def precintar(self, bulto, precinto, remito, instante):
        bulto._fijar_precinto(precinto, remito)
        bulto._cambiar_estado(Precintado(), instante, ...)

class Bulto:
    def precintar(self, precinto, remito, instante):
        return self._estado.precintar(self, precinto, remito, instante)
```

1

Contrato abstracto

EstadoBulto rechaza toda operación por defecto.

2

Cada estado decide

Redefine solo lo que admite y elige la transición.

3

El contexto delega

Bulto se pasa a sí mismo; un único _cambiar_estado registra el evento.

DOBLE VERIFICACIÓN

El stock se vende recién al final



Precinto conforme

Recibido, luego verificar contenedor y verificar remito. Verificar solo uno de los dos nunca habilita stock.

Precinto roto o distinto

Rechazado en la misma recepción: el stock pasa de EN_TRANSITO a RECHAZADO y el remito queda rechazado. Nunca fue vendible.

VENTAS Y DEVOLUCIONES

Stock y caja, en la misma operación

CV-00000001

Venta

- Varios productos; Python copia el precio vigente.
- Cobros divididos por medio de pago, que deben sumar exacto el total.
- Descuenta solo stock DISPONIBLE del local.
- Un ingreso de caja por cada cobro, por local y medio.
- Una clave de operación evita duplicar la venta si se reenvía.

DV-00000001

Devolución

- Solo de una venta registrada, en su mismo local.
- Parcial, sin superar lo vendido en cada línea.
- Apta vuelve a DISPONIBLE; no apta queda BLOQUEADO.
- Reintegro al precio histórico, entre los medios cobrados.
- La venta y su comprobante no se modifican.

Importes exactos en centavos, sin float · comprobante interno que se imprime desde el navegador.

VALIDACIONES DE NEGOCIO

Cinco reglas, dos barreras

Validación del enunciado	Regla en el dominio	Respaldo
Stock disponible solo tras verificar remito y bulto	Remito.verificar exige el bulto Verificado; recién ahí pasa a DISPONIBLE	Precinto roto: Rechazado en la misma recepción
Trazabilidad inmutable de los estados del bulto	Un único _cambiar_estado agrega un evento de solo lectura	Triggers SQL rechazan UPDATE y DELETE
No vender más que el stock disponible y verificado	Inventario.vender descuenta solo DISPONIBLE y valida todas las líneas antes	409 sin efectos parciales
Sin productos ni proveedores duplicados	Código comercial único sin distinguir mayúsculas; nunca se reutiliza	Códigos reservados con clave única en SQLite
Devoluciones solo de ventas registradas y válidas	Misma venta y local, sin superar lo vendido ni lo cobrado por medio	Triggers sobre lo devuelto acumulado

PATRÓN STRATEGY · EXTRACTO

El reporte delega cómo agrupar

```
class ReporteVentas:  # contexto
    def cambiar_estrategia(self, estrategia):
        self._estrategia = estrategia

    def generar(self, periodo, ventas, devoluciones, caja, local=None):
        for movimiento in caja.movimientos(local):
            orden, etiqueta = self._estrategia.clave(movimiento)

class PorMedioPago(EstrategiaAgrupacion):
    def clave(self, movimiento):
        return movimiento.medio.identificador, movimiento.medio.nombre

reporte = ReporteVentas(PorDia())
reporte.cambiar_estrategia(PorLocal())
```

1

¿Qué algoritmo uso?

Los mismos movimientos, por día, por local o por medio de pago.

2

Elegida desde afuera

El cliente la pasa y la intercambia, sin if por tipo en el contexto.

3

Fácil de extender

Una agrupación nueva no toca el contexto; está probado.

REPORTES

Cuatro reportes no triviales

R-01 · Strategy

Ventas, cobros e ingresos netos

Ventas menos devoluciones; cobros y reintegros agrupados por día, local o medio.

Filtros: período y local

R-02

Productos más vendidos y margen

Unidades netas, importe a precio histórico, costo de referencia y margen estimado.

Filtros: período y local

R-03

Existencias, mínimos y faltantes

Saldo por condición, mínimo, déficit y alerta cuando el disponible llega al mínimo.

Filtros: local o CD, categoría, solo en mínimo

R-04

Rechazos logísticos

Bultos rechazados por precinto, unidades y valor bloqueado al costo del bulto.

Filtros: período, local destino y producto

Solo leen operaciones confirmadas: ningún reporte modifica stock ni caja. R-02 concilia con R-01 para el mismo período y local.

PERSISTENCIA EN SQLITE

Cada operación, una transacción

SQL parametrizado

sqlite3 con parámetros ? y claves foráneas activas en cada conexión. Sin ORM.

Todo o nada

Una venta guarda líneas, cobros, stock y caja juntos. Si algo falla: rollback, y la memoria vuelve al último estado confirmado.

Segunda barrera

UNIQUE, CHECK y triggers: el historial del bulto y los movimientos de caja no se editan ni se borran.

Al iniciar, el estado se reconstruye en orden y cada fila vuelve a pasar por el dominio:

Catálogo → Compras → Reposición y logística → Ventas → Devoluciones → Inventario → Caja

PRUEBAS CON PYTEST

Cada regla tiene su prueba

766

casos aprobados

Suite completa al 01/11/2026, en 21 archivos. Cada prueba usa una base temporal: nunca la real.

Algunas, sobre el bulto y la venta

`test_circuito_completo_habilita_stock_solo_con_bulto_y_remito_verificados`

`test_precinto_roto_o_distinto_rechaza_en_la_misma_recepcion`

`test_cada_estado_rechaza_las_operaciones_que_no_admite_sin_efectos`

`test_estado_bulto_es_un_contrato_abstracto`

`test_stock_pendiente_rechazado_o_en_transito_nunca_se_vende`

`test_solo_se_vende_lo_que_llego_con_bulto_y_remito_verificados`

También: fallos de guardado sin efectos parciales, reinicio de la app, envíos concurrentes y reportes contra resultados calculados a mano.

ESTADO DEL TRABAJO

Dónde estamos hoy

Fase	Estado
F-00 · Alcance y políticas	Parcial
F-01 · Catálogo	Completa
F-02 · Persistencia, compras e inventario	Completa
F-03 · Circuito logístico con State	Completa
F-04 · Venta, cobro y comprobante	Completa
F-05 · Devoluciones	Completa
F-06 · Reportes y Strategy	En curso
F-07 · Integración y entrega	Pendiente

Próximos pasos

- API y pantalla de los reportes
- Elegir la salida: PDF con ReportLab o CSV
- Escenarios integrados, revisión UML y entrega
- Consultas al docente: rúbrica, borrado físico y un remito por bulto

DEMO

El recorrido de un bulto, en vivo

01
Compra al proveedor: stock en el CD

02
Necesidad del local y orden de reposición

03
Preparar el bulto: reserva en el CD

04
Precintar: se emite el remito

05
Despachar hacia el local

06
Recibir con precinto roto: Rechazado

07
Recibir otro bulto con precinto intacto

08
Verificar bulto y remito: disponible

09
Vender con cobro dividido y comprobante

10
Devolver una unidad no apta: bloqueada

Cada paso se ve en Stock por ubicación y en la caja del local, con su documento de origen.

GRUPO 95 · TEMA 3

Gracias

¿Preguntas?

State

El ciclo de vida del bulto, con historia inmutable.

Strategy

Agrupaciones intercambiables para el reporte de ventas.

Reglas en Python

Dominio que valida todo, con SQLite como segunda barrera.